PLATFORM BUILT IN

lifepal

PRESENTED TO

TRIPADVISOR

DATA PLATFORM · SENIOR DATA ENGINEER

DATA ENGINEERING · FIRESIDE CHAT · NEXUS BEST PRACTICES FROM LIFEPAL

Nexus

Kafka in, Flink in the middle, BigQuery out. Every event counted once, on time, and checked

Four years leading the data platform at Lifepal, a high-traffic consumer marketplace: Kafka and Debezium streams next to batch sources, a config-driven framework other engineers built on, SLAs, 500+ quality checks and a root-cause review after every incident. This pack applies those practices to a travel site's data on Tripadvisor's listed stack, Kafka → stream processing → Spark → Snowflake and BigQuery, in a runnable repository with synthetic data. Every run figure comes from that run; every Lifepal figure is from my CV.

PRESENTED BY
R. Firman Insan Muhammad (Vim)
Senior Data Engineer · London

TO
Data Platform
Tripadvisor

ROLE
Senior Data Engineer
London, remote / hybrid

STACK
Kafka · Flink · BigQuery
Spark · Snowflake · Java · Python

CONTENTS

What I'd like to cover

Ten minutes on what I've built, then the parts of the role I'd own first. Every page points to working code.

01	**Measured outcomes**	What changed at Lifepal, in numbers
02	**The role, line by line**	Where I've done it before, and where I haven't
03	**Six streaming problems**	Planted in the topics, caught in the run
04	**The stream job**	Event time, watermarks, lateness, dead letters
05	**Stream first, batch checks**	Reconciliation, record for record; Spark agrees
06	**Snowflake and BigQuery from one contract**	Governance as generated code
06	**Modeling and governance**	Grain, keys, lineage, policy tags, quality gates, erasure
07	**SLAs and an incident**	Detection, root cause, postmortem
08	**Apache Flink**	The job on Flink 1.20: topology, state, watermarks, production on Kafka
08	**Java**	A twin of the job, tested against the reference
10	**Why this, not that**	Each decision re-run against its alternatives
11	**My first 90 days**	How I'd start in Data Platform

What changed at Lifepal

Figures from my CV and production work, described at the level I can share publicly.

99.7%
reporting latency cut: 24 h → under 5 min

75%
nightly batch cut: 6 h → 90 min by tuning Spark and SQL

$11k
monthly platform cost against a $19k forecast

−60%
data-trust escalations: SLAs, 500+ checks, anomaly holds

−40%
duplicate records

How: one governed lakehouse out of fragmented batch jobs and legacy ingestion; Kafka + Debezium CDC next to batch from APIs and partner files; a config-driven Python framework; Kimball models and a dbt semantic layer for five teams.

Team: led 3–5 engineers; two promoted to Senior; onboarding halved from 4 weeks to 2. An MCP server for governed agent access and a RAG copilot with 1,000+ outcomes a day.

How my experience maps to the role

The job description's asks, what I've done, and where it shows in this repository.

THE ROLE ASKS FOR	WHAT I'VE DONE	HERE
ETL/ELT across Snowflake and BigQuery	Snowflake + dbt at Deloitte; Databricks + dbt at Lifepal	warehouse/ · warehouse.py
Own pipelines: SLAs, monitoring, root cause	SLAs, freshness and volume alerts, runbooks, RCA after every incident	sla.py · oncall.html
Streaming and batch: Flink, Spark, Kafka	Kafka + Debezium with Spark; 6 h → 90 min batch	flink/job · stream.py · spark_batch.py
Production Java and Python	Python for ten years; Java is my ramp	java/WindowJob.java
Shared frameworks, governance tooling	Config-driven ingestion; PII masking, RBAC, lineage	contracts/ · governance.sql
GenAI, analytics, ML	RAG copilot; MCP server; feature pipelines	Fireside story
Flink, specifically	Not in production. Here: real Flink 1.20 in CI, identical windows to the Python job	flink/job/

What Kafka really delivers

A week of a travel site: 26,426 records on 13 partitions. Each problem is planted, handled and tested.

PROBLEM	IN THE RUN	WHAT HANDLES IT
SDK retries	112 events re-sent, same event_id, new offset	Dedupe on id, 24 h checkpointed state: 112 dropped
An offline upload	192 iOS events, 2–8 hours late	Side output past an hour's lateness; the batch explains 159 windows
A schema change	app 7.4: price {amount, currency}, 302 records	One reader upcasts v1 and v2; unknown currency is a dead letter
A poison pill	partition 3, offset 171: 57 bytes of truncated JSON	74 restarts, paged at 14:35; fixed with the DLQ
A scraper	1,515 views in an hour that has 29.0	Hour held (2σ and ×2); 3 users removed; released
Erasure, then a stale version	an outbox replay re-sends version 3 after the tombstone	Version rule: 1 rejected; deleted in both warehouses

Event time, decided by the watermark

The watermark is the minimum over active partitions, less the out-of-orderness. A quiet partition stops holding it back; when every partition is quiet, processing time carries it.

$$\text{wm} = \max(\text{wm}, \min_{\text{active p}} \text{max_event(p)} - \text{ooo}) \cdot \text{fire} \iff \text{end} \le \text{wm} \cdot \text{drop} \iff \text{end} + \text{lateness} \le \text{wm}$$

SRC/NEXUS/STREAM.PY — FLINK SEMANTICS

```
    def _watermark(self, now: int) -> int:
        if self.wm_mode == "global_max":    # one clock for the topic: the newest event
anywhere
            return max(self.s.wm, max(self.s.part_max.values()) - self.ooo) if
self.s.part_max else self.s.wm
        if self.wm_mode == "min_all":       # every partition counts, quiet or not: a quiet
one holds everything back
            return max(self.s.wm, min(self.s.part_max.values()) - self.ooo) if
self.s.part_max else self.s.wm
        active = [self.s.part_max[p] for p, seen in self.s.part_seen.items() if now - seen
< IDLE]
        if active:
            wm = min(active) - self.ooo
        elif self.s.part_max:           # every partition idle: let processing time carry
the watermark (a heartbeat)
            wm = now - self.ooo - IDLE
        else:
            wm = NEG
        return max(self.s.wm, wm)

    def _emit(self, key, w: dict, now: int):
        value = {"n": w["n"], "sum": round(w["sum"], 4)}
        old = self.sink.get(key)
        self.sink[key] = {"value": value, "first_emit": old["first_emit"] if old else now,
"last_emit": now,
                          "emits": (old["emits"] if old else 0) + 1}
        self.emits.append((now, key[0] + self.size))

    def _advance(self, now: int):
        wm = self._watermark(now)
        if wm <= self.s.wm and self.s.wm != NEG:
            return
        self.s.wm = wm
```

THE RULES

- **Fire** when the watermark passes a window's end; **re-fire** on a late record within the hour; past it, the **side output**.
- **Dedupe** on event_id, kept 24 hours. **Dead letters** for what can't be read, with the reason.
- Offsets, windows, dedupe state and the ledger are **checkpointed together**; sinks are keyed, so a restore re-sends without changing anything.

IN THE RUN

- A TaskManager lost at 09:17: restored from the 09:10 checkpoint, **55** windows re-sent, sink **identical**.
- Ledger, page_views: read 22,813 = 22,509 counted + 112 duplicates + 192 side output.

lifepal

Dedupe, lateness, side output: the code

One function decides every record: read it, drop a retry, count it, re-fire a late window, or set it aside for the batch.

SRC/NEXUS/STREAM.PY — PYTHON · THE REFERENCE

```
    def _process(self, rec: Record, now: int):
        st = self.s
        st.ledger["read"] += 1
        try:
            v = upcast(self.name, rec.raw)
        except BadRecord as e:
            self._note(rec, None, "dead letter" if self.dlq_enabled else "job fails", now,
reason=str(e), raw=rec.raw)
            if not self.dlq_enabled:
                raise Poison(rec, str(e)) from None
            st.ledger["dead_letters"] += 1
            self.dlq[(rec.partition, rec.offset)] = str(e)
            return
        rid = v[self.t["id"]]
        if rid in st.seen:
            st.ledger["duplicates"] += 1
            self._note(rec, rid, "duplicate", now, first_seen=iso(st.seen[rid]))
            return
        st.seen[rid] = now
        et = parse(v["event_time"])
        st.part_max[rec.partition] = max(st.part_max.get(rec.partition, NEG), et)
        st.part_seen[rec.partition] = now
        if not self.windowed:
            self._apply_version(v, now, rec)
        else:
            start = et - et % self.size
            key = (start, tuple(v[g] for g in self.group_by))
            if start + self.size + self.lateness <= st.wm:
                st.ledger["late_dropped"] += 1
                self._note(rec, rid, "too late: side output", now,
event_time=v["event_time"], window=iso(start),
```

WHAT IT DECIDES

- **112** retries dropped by id; **16** late re-fires; **192** sent to the side output.
- A record that can't be read raises **Poison** unless the dead-letter topic is on: the incident in one line.

CHECKED THREE WAYS

- The Flink job and the Java twin reach the same windows; the batch explains every difference.

The batch that checks the stream

Every record from the topic archive: parse both schema versions, keep the first arrival of each id, window by event time.

SRC/NEXUS/SPARK_BATCH.PY — PYSPARK 3.5.4

```
def recompute(spark: SparkSession, name: str) -> dict:
    t = topic(name)
    prod = t["product"]
    size = prod["window_minutes"] * 60_000
    raw = (spark.read.json(str(LANDING / "kafka" / name / "p*.jsonl"))
           .withColumn("partition", F.regexp_extract(F.input_file_name(), r"p(\d+)\.jsonl",
1).cast("int"))
           .withColumn("v", F.from_json("value", VALUE)))
    rates = F.create_map(*[x for k, r in fx().items() for x in (F.lit(k), F.lit(r))])
    parsed = (raw.withColumn("ok", F.col("v.event_time").isNotNull())
              .withColumn("id", F.col(f"v.{t['id']}"))
              .withColumn("et", F.unix_millis(F.to_timestamp("v.event_time", "yyyy-MM-
dd'T'HH:mm:ss.SSSX")))
              .withColumn("revenue_gbp", F.coalesce(F.col("v.cpc_gbp"),
                                                    F.round(F.col("v.price.amount") *
rates[F.col("v.price.currency")], 4))))
    bad = parsed.filter(~F.col("ok")).count()
    first = Window.partitionBy("id").orderBy("ts", "partition", "offset")
    events = (parsed.filter("ok").withColumn("rn", F.row_number().over(first)).filter("rn =
1")
              .withColumn("window_start", F.col("et") - F.pmod(F.col("et"), F.lit(size))))
    group = [F.col(f"v.{g}").alias(g) for g in prod["group_by"]]
    measure = F.sum("revenue_gbp") if any(h == "sum" for h in prod["measures"].values())
else F.lit(0.0)
    rows = events.groupBy("window_start", *group).agg(F.count("*").alias("n"),
measure.alias("sum")).collect()
    return {"windows": {(r["window_start"], tuple(r[g] for g in prod["group_by"])): {"n":
r["n"], "sum": round(r["sum"] or 0.0, 4)}
                        for r in rows},
            "bad_records": bad, "events": events.count()}
```

IN THE RUN

- page_views: **8,764** windows · partner_clicks: **984** · identical to the plain-Python batch.
- The poison pill is a parse failure here too: counted, never guessed.

AT LIFEPAL

- Spark and SQL tuning took the nightly batch from 6 hours to 90 minutes and kept spend at $11k against a $19k forecast.

Fast first. Right by morning.

The nightly batch recomputes every window from the whole topic. Every difference must be explained by the stream's side output.

TOPIC	WINDOWS	EQUAL	EXPLAINED	UNEXPLAINED	STREAM	BATCH	SIDE OUTPUT	SPARK AGREES
page_views	8,764	8,605	159	**0**	22,509	22,701	192	identical
partner_clicks	984	983	1	**0**	1,245	1,246	1	identical

The equation the run checks: batch events − stream events = side output, for every topic and every window. The PySpark batch (Spark 3.5.4) produces the same windows as the plain-Python one.

Volume hold: 2026-10-10 03:00 had 1,515 views where 29.0 is usual (z = 420.3). Held from Gold; three users over 120 views an hour removed; re-checked; released.

Write it once. Mask it everywhere.

contracts/sources.yml names the personal fields. The DDL, MERGE, masking and erasure for both warehouses are generated from it and committed; CI fails if they drift.

WAREHOUSE/SNOWFLAKE/REVIEWS_CURRENT.SQL — GENERATED

```
MERGE INTO NEXUS.GOLD.REVIEWS_CURRENT t
USING NEXUS.STAGE.REVIEWS_CURRENT s
  ON t.review_id = s.review_id
WHEN MATCHED AND s.erased AND s.version > t.version THEN DELETE
WHEN MATCHED AND NOT s.erased AND s.version > t.version THEN UPDATE SET location_id =
s.location_id, rating = s.rating, status = s.status, lang = s.lang, version = s.version,
_loaded_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED AND NOT s.erased THEN INSERT (review_id, location_id, rating, status,
lang, version, _loaded_at)
  VALUES (s.review_id, s.location_id, s.rating, s.status, s.lang, s.version,
CURRENT_TIMESTAMP());
```

SAME PRODUCT, TWO DIALECTS

- **Layout:** CLUSTER BY (TO_DATE(window_start), partner) · PARTITION BY DATE(window_start) CLUSTER BY partner
- **Personal data:** masking policy, SHA2 unless PII_READER · policy tag + data masking rule
- **Erasure list:** FLATTEN(PARSE_JSON(:review_ids)) · UNNEST(@review_ids)

CHECKED, NOT ONLY RENDERED

- Gold loaded with the same upsert semantics; **11/11** checks pass, including masking for every personal field and the erasure (5 rows removed).

AT LIFEPAL

- PII masking, role-based access and lineage in one governed layer; certified models behind an MCP server for agents.

Grain first. Rules travel with the data.

Kafka topics land in raw tables that keep every field and where it came from; Gold products state their grain and key; one contract names the personal fields, the rules and the SLAs.

GOLD PRODUCT	GRAIN	KEY (THE MERGE)	BIGQUERY LAYOUT	ROWS	SLA
location_views_hourly	one row per location per hour	window_start, location_id	PARTITION BY DATE(window_start) CLUSTER BY location_id	8,755	tier 2 · 90 min
partner_revenue_15m	one row per partner per 15 minutes	window_start, partner	PARTITION BY DATE(window_start) CLUSTER BY partner	984	tier 1 · 45 min
reviews_current	one row per live review (latest version)	review_id	CLUSTER BY location_id	1,120	tier 2 · 60 min

Personal data

raw_page_views.user_hash, raw_reviews.reviewer_hash, raw_reviews.reviewer_email: a policy tag in BigQuery and a masking policy in Snowflake, from one line of the contract; a check per field.

Quality gates

11/11 checks pass: rules per product from quality.yml, masking, and the erasure. A volume gate held the scraper hour before Gold.

Erasure and schema change

A tombstone deletes the review in raw and Gold in both warehouses (5 rows). Schema v2 is flattened into raw; Gold keeps its grain.

One MERGE, generated for both warehouses

The key from the contract becomes the MERGE condition; a late re-fire updates one row; a tombstone deletes.

SRC/NEXUS/WAREHOUSE.PY — RENDERS BIGQUERY + SNOWFLAKE

```
def _merge(d: str, table: str, spec: dict) -> str:
    cols, key = list(spec["cols"]), spec["key"]
    tgt, src = _name(d, "gold", table), _name(d, "stage", table)
    now = "CURRENT_TIMESTAMP()"
    on = " AND ".join(f"t.{k} = s.{k}" for k in key)
    vals = [c for c in cols if c not in key]
    changed = " OR ".join(f"t.{c} IS DISTINCT FROM s.{c}" for c in vals)
    sets = ", ".join(f"{c} = s.{c}" for c in vals)
    ins = ", ".join(cols)
    sql = f"MERGE INTO {tgt} t\nUSING {src} s\n  ON {on}\n"
    if "version" in spec:
        v, tomb = spec["version"], spec["tombstone"]
        sql += (f"WHEN MATCHED AND s.{tomb} AND s.{v} > t.{v} THEN DELETE\n"
                f"WHEN MATCHED AND NOT s.{tomb} AND s.{v} > t.{v} THEN UPDATE SET {sets},
_loaded_at = {now}\n"
                f"WHEN NOT MATCHED AND NOT s.{tomb} THEN INSERT ({ins}, _loaded_at)\n
VALUES ({', '.join('s.' + c for c in cols)}, {now});\n")
    else:
        sql += (f"WHEN MATCHED AND ({changed}) THEN UPDATE SET {sets}, _loaded_at =
{now}\n"
                f"WHEN NOT MATCHED THEN INSERT ({ins}, _loaded_at)\n  VALUES ({',
'.join('s.' + c for c in cols)}, {now});\n")
    return sql
```

GENERATED, NOT HAND-WRITTEN

- **16** SQL files from one contract; CI fails if they drift.

GOVERNANCE IN THE SAME FILE

- Personal fields get a policy tag in BigQuery and a masking policy in Snowflake; erasure SQL is generated for both.

partner_revenue_15m late for 79 minutes

Freshness is measured from the completeness the job publishes, so a quiet night is not an incident. One thing was all week.

UTC	WHO	WHAT
14:05	job	Record 3/171 can't be parsed. The job fails, restores from its checkpoint, reads the same record, and fails again.
14:08	would page	Restart count passes 3 in 10 minutes. Not alerted on yet: this is action 2.
14:35	SLA monitor	partner_revenue_15m is 50 minutes old, over its 45-minute SLA. Page sent.
14:39	on call	Acknowledged. Job RESTARTING; exception points to partition 3 offset 171.
15:20	on call	Mitigation: deploy with the dead-letter topic on. Restore from the last checkpoint; the bad record goes to the DLQ.
15:25	SLA monitor	Back within SLA. Every delayed window re-computed from the checkpoint.

29 min

to detect: freshness had to cross 45 minutes first

3 min

with a restart alert: 74 restarts said it early

Root cause: a record the job couldn't parse was treated as a reason to stop; a restart reads the same offset again. **Fix:** dead-letter by default in the shared template (done), page on restarts, alert on DLQ depth, a producer contract test.

Error budget for the week: 2.0 late samples allowed at 99.9%; the incident used 10. 11 windows late, none wrong; one record in the DLQ.

Freshness, measured like a consumer feels it

Age is now minus the completeness the job publishes. Sampled every five minutes; the incident record is built from the same samples.

SRC/NEXUS/SLA.PY — PYTHON

```python
def age_series(job, start: int, end: int) -> list[tuple[int, float]]:
    emits = sorted(job.progress)
    out, i, visible = [], 0, None
    for t in _samples(start, end):
        while i < len(emits) and emits[i][0] <= t:
            visible = emits[i][1] if visible is None else max(visible, emits[i][1])
            i += 1
        if visible is not None:
            out.append((t, round((t - visible) / MIN, 1)))
    return out


def lag_series(job, start: int, end: int) -> list[tuple[int, float]]:
    """Minutes the oldest record that has arrived but isn't processed yet has been waiting
(0 when caught up)."""
    recs = sorted(zip((r.ts for r in job.records), job.done_at))
    out, i, waiting = [], 0, []
    for t in _samples(start, end):
        while i < len(recs) and recs[i][0] <= t:
            heapq.heappush(waiting, recs[i])
            i += 1
        while waiting and waiting[0][1] <= t:
            heapq.heappop(waiting)
        # a record processed later than an older one still waiting stays in the heap: drop
finished ones lazily
        live = [w for w in waiting if w[1] > t]
        out.append((t, round((t - min(w[0] for w in live)) / MIN, 1) if live else 0.0))
    return out
```

IN THE RUN

- partner_revenue_15m: on time **99.50%**; one breach, 79 minutes.
- Paged at 14:35; a restart alert would have paged at 14:08.

WHY COMPLETENESS

- Measuring from the newest non-empty window would page every quiet night.

The same job, on Flink

Parse with a dead-letter output, dedupe in keyed state, per-partition watermarks, event-time windows with allowed lateness, a side output for what's too late.

records → Parse ⟶ DLQ · keyBy(id) → Dedupe(TTL 24 h) · keyBy(group) → Tumbling(1 h) → allowedLateness(1 h) ⟶ late

FLINK/JOB/.../NEXUSJOBS.JAVA — FLINK 1.20.1 · JAVA 17

```
    public static Outputs build(DataStream<KafkaRecord> records, Product p, boolean
replayWatermarks) {
        SingleOutputStreamOperator<Event> parsed = records.process(new
Parse(p.topic())).name("parse " + p.topic());
        SingleOutputStreamOperator<Event> unique = parsed.keyBy(e -> e.id).process(new
Dedupe()).name("dedupe on id");
        final long ooo = p.outOfOrderness().toMillis();       // a long, not the record:
the lambda is serialized
        DataStream<Event> timed = replayWatermarks
                ? unique.assignTimestampsAndWatermarks(WatermarkStrategy
                        .<Event>forGenerator(ctx -> new PartitionWatermarks(ooo))
                        .withTimestampAssigner((e, ts) -> e.eventTime)).name("watermarks")
                : unique;
        SingleOutputStreamOperator<WindowRow> windows = timed
                .keyBy(e -> e.group)
                .window(TumblingEventTimeWindows.of(p.window()))
                .allowedLateness(p.lateness())
                .sideOutputLateData(TOO_LATE)
                .aggregate(new Windows.CountSum(), new Windows.Emit())
                .name(p.name());
        return new Outputs(windows, parsed.getSideOutput(Parse.DEAD_LETTERS),
unique.getSideOutput(Dedupe.DUPLICATES),
                windows.getSideOutput(TOO_LATE));
    }
}
```

ON A MINICLUSTER, IN CI

- **8,707** windows, **identical** to the Python job: 112 duplicates, 192 too late, 8,723 firings.
- partner_clicks: the poison pill **dead-lettered** (partition 3, offset 171), revenue windows **identical** to the batch.

ONE HONEST DIFFERENCE

- Python carries the watermark on processing time when every partition is quiet; Flink's idleness doesn't. So at 3 a.m. Flink kept one late click the Python job sent to the side output. Both are explained.

STATED PLAINLY

- Real Flink, run locally, not on a production cluster. At Lifepal my streaming was Kafka + Debezium with Spark.

On Kafka, with the incident's lessons built in

NexusKafka runs the same topology from KafkaSource to transactional Kafka sinks. Compiled in CI; there is no cluster behind a portfolio.

FLINK/JOB/.../NEXUSKAFKA.JAVA — KAFKASOURCE · KAFKASINK

```
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();
        env.enableCheckpointing(Duration.ofMinutes(10).toMillis(),
CheckpointingMode.EXACTLY_ONCE);

env.getCheckpointConfig().setMinPauseBetweenCheckpoints(Duration.ofMinutes(1).toMillis());
        // 3 restarts in 10 minutes, then fail and page: a poison pill never crash-loops
for 74 minutes again
        env.setRestartStrategy(RestartStrategies.failureRateRestart(3, Time.minutes(10),
Time.seconds(30)));

        KafkaSource<KafkaRecord> source = KafkaSource.<KafkaRecord>builder()
                .setBootstrapServers(brokers)
                .setTopics(p.topic())
                .setGroupId("nexus-" + p.name())
                .setStartingOffsets(OffsetsInitializer.committedOffsets())
                .setDeserializer(KafkaRecordDeserializationSchema.of(new RecordSchema()))
                .build();
        // per-partition watermarks and idleness come from the source itself, one generator
per split
        WatermarkStrategy<KafkaRecord> wm = WatermarkStrategy.
<KafkaRecord>forBoundedOutOfOrderness(p.outOfOrderness())
                .withIdleness(Duration.ofMinutes(5))
                .withTimestampAssigner((r, kafkaTs) -> eventTime(r, kafkaTs));

        Outputs o = NexusJobs.build(env.fromSource(source, wm, "kafka " + p.topic()), p,
false);
        o.windows().map(NexusKafka::row).sinkTo(sink(brokers, "gold." +
p.name())).name("upsert gold." + p.name());
        o.deadLetters().map(NexusKafka::dead).sinkTo(sink(brokers, "dlq." +
p.topic())).name("dlq." + p.topic());
        o.tooLate().map(e -> e.id + "," + e.eventTime).sinkTo(sink(brokers, "late." +
p.topic())).name("late." + p.topic());
```

WHAT THE INCIDENT CHANGED

- **Dead letters** to `dlq.partner_clicks`, never a crash loop.
- **failureRateRestart(3, 10 min)**: three restarts, then fail and page. 74 restarts can't happen again.
- **Exactly-once** checkpoints every 10 minutes and transactional sinks: a restore re-sends nothing twice.

WATERMARKS

- Per split from the source: **forBoundedOutOfOrderness(ooo).withIdleness(5 min)**, so a quiet partition can't hold the job back.

Python first. Java, checked.

The page_views job, in plain JDK 17 with no framework. CI compiles it, runs it on the same week, and fails if a single window differs from the Python job.

JAVA/SRC/NEXUS/WINDOWJOB.JAVA — JDK 17 · NO DEPENDENCIES

```
    void process(Rec r, long now) {
        count("read");
        Map<String, Object> v = Json.object(r.value());
        String id = (String) v.get("event_id");
        if (seen.containsKey(id)) {
            count("duplicates");
            return;
        }
        seen.put(id, now);
        long et = Instant.parse((String) v.get("event_time")).toEpochMilli();
        partMax.merge(r.partition(), et, Math::max);
        partSeen.put(r.partition(), now);
        long start = et - Math.floorMod(et, SIZE);
        Key key = new Key(start, (String) v.get("location_id"));
        if (start + SIZE + LATENESS <= wm) {
            count("late_dropped");
        } else {
            Window w = windows.computeIfAbsent(key, x -> new Window());
            w.n++;
            count("accepted");
            if (w.fired) {
                count("late_updates");
                emit(key, w);
            }
        }
        advance(now);
```

IN THE RUN

- **8,707** windows, **identical** to the Python job, in 731 ms.

STATED PLAINLY

- Kafka, Spark and Python are production experience. At Lifepal the streaming was Kafka + Debezium with Spark, not Flink. Snowflake is from Deloitte; BigQuery I haven't run in production, so its SQL here is mapped from Snowflake and Databricks.
- Java is the part I'd ramp up on. This is how I'd start: the same semantics, tested against a reference I trust.

WHY THIS, NOT THAT

How each decision was reached

Every choice below was re-run against its alternatives on the same week of Kafka records, and measured. Where the data says a choice is oversized, the slide says so.

DECISION	CHOSEN	ALTERNATIVES RE-RUN	METHOD
Watermark	min over active partitions, idle skipped	wait for every partition · one global clock	bench · compare
Out-of-orderness	10 min	0 · 2 · 30 min	bench
Allowed lateness	60 min	0 · 15 · 180 min	bench
Dedupe state	24 h TTL	36 s · 15 min · 1 h	bench
Bad records	dead-letter topic	fail and restart · skip silently	replay · compare
Alerting	restarts and freshness	freshness only	replay
Freshness metric	published completeness	newest non-empty window	trial
Volume hold	2σ and ×2	2σ · 3σ · ×2 only	trial · bench
Warehouse layout	day partitions, cluster by partner	none · day only	bench (modelled)
Engine	Flink, checked by Python, Java and Spark	one engine, unchecked	bench

METHOD Benchmark sweep: the same week re-run per option · Trial and error: built, measured, changed · Failure replay: the incident re-run each way · Comparison with how other engines do it

Which clock says an hour is over

Three watermarks on the same week of page_views. The one that waits for every partition stalls whenever one goes quiet.

WATERMARK	COUNTED	MISSED	FRESH P95	WORST	SLA BREACHES
Min over active partitions ✔	22,509	192	70 min	75 min	0
Min over every partition	22,509	192	145 min	245 min	362
One global clock (max)	22,509	192	70 min	85 min	0

OUT-OF-ORDERNESS	LATE RE-FIRES	FRESH P50	FRESH P95	SLA BREACHES
0 min	20	35 min	60 min	0
2 min	19	35 min	60 min	0
10 min ✔	16	45 min	70 min	0
30 min	4	65 min	90 min	3

HOW I GOT HERE

- Benchmark sweep: the same week re-run per option
- Comparison with how other engines do it

WHAT THE NUMBERS SAY

- Waiting for every partition: **362** breach samples, p95 145 min. Quiet partitions hold the whole job back.
- A global clock costs nothing here because the partitions stay in step. It is the one that drops records when a partition lags, after a restore for example.

ELSEWHERE

- **Flink**: per-split watermarks, the minimum wins, `withIdleness`. **Spark Structured Streaming**: one watermark from the max event time. **Kafka Streams**: stream time per task, plus a grace period.

WHAT I'D REVISIT

- 10 min of out-of-orderness is headroom for phone clocks. The data says 2 min would be 10 min fresher at p95 for 3 more re-fires a week.

How long to wait, how long to remember

Allowed lateness trades events counted in the stream against state held open. Dedupe TTL trades retries caught against keys kept.

ALLOWED LATENESS	COUNTED	MISSED (BATCH HAS IT)	RE-FIRES	PEAK OPEN WINDOWS
0 min	22,489	212	0	87
15 min	22,500	201	10	87
60 min ✔	22,509	192	16	170
180 min	22,561	140	49	325

DEDUPE TTL	RETRIES CAUGHT	PEAK KEYS HELD
36 s	112	26
15 min	112	373
1 h	112	1,514
24 h ✔	112	5,099

HOW I GOT HERE

- Benchmark sweep: the same week re-run per option
- Comparison with how other engines do it

WHAT THE NUMBERS SAY

- 60 min is the smallest lateness that absorbs all organic lateness: what's left (192) is the offline upload, hours late. 180 min counts 52 more but holds 1.9× the state; the nightly batch already counts them.
- Retries arrive within 20 s (p50 11 s), so 15 minutes would catch all 112. 24 h is a deliberate margin for re-delivery after an outage: 5,099 keys, well under a megabyte.

ELSEWHERE

- **Flink** `allowedLateness` and keyed state TTL; **Kafka Streams** `grace()`; **Spark** `dropDuplicatesWithinWatermark`. Kafka's idempotent producer only dedupes broker-level retries, not an app re-sending.

WHAT I'D REVISIT

- Size the TTL from the longest consumer rewind the team has actually done, not from retries.

Stop, skip, or keep it aside

The poison pill replayed three ways on the same partner_clicks week, and the two alerts that could have paged.

POLICY	RESTARTS	PRODUCT LATE	WINDOWS LATE	RECORD KEPT
Fail and restart	74	79 min	11	1
Skip silently	0	0 min	0	0
Dead-letter topic ✔	0	0 min	0	1

ALERT	TIME TO PAGE
Freshness over the SLA	29 min
Restarts > 3 in 10 min, too ✔	3 min

HOW I GOT HERE

- Failure replay: the incident re-run each way
- Comparison with how other engines do it

WHAT THE NUMBERS SAY

- Failing cost 74 restarts and 79 minutes on a tier-1 product for one record. Skipping costs nothing now and loses the evidence. Dead-lettering costs nothing and keeps it.
- Freshness is a lagging signal: 29 minutes. The restart count said the same in 3.

ELSEWHERE

- **Kafka Connect**: `errors.tolerance=all` with a DLQ topic. **Flink**: a side output, as here. **Spark**: permissive parsing with a corrupt-record column.

AT LIFEPAL

- Every incident ended with a root-cause review and a change to the platform; data-trust escalations fell 60%.

Built, measured, changed

Both started as the obvious version. The run showed what it would page for, so both changed. This is the trial and error, kept.

FRESHNESS MEASURED FROM	PAGES IN A WEEK	LATE SAMPLES	WORST
Newest non-empty window (first version)	38	191	230 min
Published completeness ✔	0	0	25 min

VOLUME HOLD	HOURS HELD	CAUGHT THE SCRAPER	FALSE HOLDS
2σ (first version)	29	yes	28
3σ	13	yes	12
2σ and ×2 ✔	1	yes	0
×2 only	1	yes	0

HOW I GOT HERE

- Trial and error: built, measured, changed · Benchmark sweep: the same week re-run per option

WHAT THE NUMBERS SAY

- With no incident at all, the first freshness metric paged **38** times in a week: a quiet 3 a.m. has no clicks, not a stale product.
- 2σ alone held **28** innocent hours: a week of history makes σ small and Saturdays look odd. Adding ×2 left only the scraper.

ELSEWHERE

- **dbt source freshness** checks the newest `loaded_at`: the first version here, fine for busy sources. Seasonal anomaly tools compare the same hour on earlier days, as this rule does.

WHAT I'D REVISIT

- ×2 alone also works on this week. σ stays to protect low-volume hours, where a jump from 2 to 5 is noise; at Lifepal the baseline was 30 days.

Read less. Check twice.

Bytes a query scans in BigQuery, modelled from the run's own row counts (48 bytes a row, an assumption), and the same week on four engines.

REVENUE FOR ONE PARTNER ON ONE DAY	BYTES SCANNED	VS NONE
No partitioning	47,232	1×
Partition by day	6,747	7× less
Day partitions, cluster by partner ✔	1,125	42× less

ENGINE	THE WEEK	OUTPUT
Python reference	2.33 s	identical
Java twin (JDK)	0.73 s	identical
Apache Flink 1.20 (MiniCluster) ✔	3.54 s	identical
PySpark batch (both topics)	12.2 s	identical

HOW I GOT HERE

- Benchmark sweep: the same week re-run per option
- Comparison with how other engines do it

WHAT THE NUMBERS SAY

- Partitioning and clustering cut the scan 42×. At this size BigQuery's 10 MB minimum dominates; on a petabyte table, the ratio is the bill.
- Four engines, one answer. Flink is slowest on a laptop-sized week: start-up dominates. It is chosen for state, checkpoints and per-split watermarks at scale, not for this benchmark.

ELSEWHERE

- **BigQuery**: partition and cluster, billed on bytes scanned. **Snowflake**: micro-partitions and clustering keys, billed on warehouse time.

AT LIFEPAL

- The same discipline took the nightly batch from 6 hours to 90 minutes and kept spend at $11k against a $19k forecast.

How I'd start in Data Platform

Listen first, then own something small end to end, then make the template better for everyone.

DAYS 1–30

Learn the estate

- Read the SLAs and the last ten incidents
- Shadow on-call; map pipelines to owners
- Ship a small fix in Java and one in Python

DAYS 31–60

Own a pipeline

- Take one tier-1 pipeline end to end
- Freshness from completeness, not last write
- A runbook a new joiner can follow

DAYS 61–90

Improve the template

- Dead letters and restart alerts by default
- Contract → Snowflake and BigQuery SQL
- Stream/batch reconciliation as a standard check

QUESTIONS I'D BRING Which pipelines page most, and why? How are SLAs measured today? Where do Snowflake and BigQuery each fit? How much of streaming is Flink vs Spark?